



Laporan Tugas Aktivitas P1

diajukan untuk memenuhi tugas mata kuliah Pemrograman Website Kelas A

LAPORAN

Oleh

Thalia Salsabilla K

(242410101064)

KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI

FAKULTAS ILMU KOMPUTER

UNIVERSITAS JEMBER

JEMBER

2025/2026

1. Buat section yang menampilkan data cuaca kota Jember menggunakan API publik <https://wttr.in/Jember?format=j1> (gratis, tidak perlu API key). Tampilkan: nama kota, suhu saat ini (dalam °C), dan deskripsi cuaca. Gunakan `async/await` dan tampilkan loading indicator saat data sedang diambil.

Disini saya menambahkan data cuaca pada dashboard admin tepatnya pada page-header untuk membantu admin dalam mengambil keputusan pengiriman pesanan ke kurir.

```
<div class="page-header">
  <div>
    <p class="greeting" id="greeting">Halo, Selamat Pagi 🌞</p>
    <h1 class="page-title">Halo, <span class="highlight">{{ auth()->user()->name }}</span>!</h1>
    <p class="page-sub">Berikut ringkasan penjualan toko Alia Cookies hari ini.</p>

    <div id="weather-section" style="margin-top: 15px; display: inline-flex; align-items: center; background: rgba(255,255,255,0.5); padding: 10px; border-radius: 10px;">
      <div id="loading-cuaca">Memuat info logistik Jember...</div>
      <div id="konten-cuaca" style="display: none; align-items: center; gap: 8px;">
        <span id="weather-icon">🌤️</span>
        <strong>Jember:</strong> <span id="suhu-saat-ini">--</span>°C,
        <span id="deskripsi-cuaca">--</span>
        <span style="margin: 0 4px; opacity: 0.5;">|</span>
        <span id="rekendasi-toko" style="font-style: italic; opacity: 0.9;"></span>
      </div>
    </div>
  </div>
  <div class="page-header-img">
    
  </div>
</div>
```

Lalu saya menambahkan kode untuk javascript untuk mengatur logika pengambilan API cuaca kota jember. Disini saya menggunakan fungsi `async/await` pada JavaScript untuk menembak *endpoint* API publik <https://wttr.in/Jember?format=j1> tanpa memerlukan API key. Agar halaman utama tidak *lag* atau terhenti (*blocking*) saat menunggu balasan dari server cuaca, digunakan fungsi asinkron dengan menampilkan tulisan *loading indicator* sementara ("Memuat info logistik..."). Setelah data JSON berhasil diterima, JavaScript akan mengambil spesifik data `temp_C` (suhu) dan `weatherDesc` (deskripsi), lalu mencetaknya langsung ke dalam elemen HTML di bagian page-header.

```

async function ambilDataCuaca() {
const loading = document.getElementById('loading-cuaca');
const konten = document.getElementById('konten-cuaca');

try {
// Ambil data API publik untuk Jember dengan format JSON (j1)
const response = await fetch('https://wttr.in/Jember?format=j1');
if (!response.ok) throw new Error('Gagal memuat API');

const data = await response.json();

// Ambil data suhu dan deskripsi cuaca
const suhu = data.current_condition[0].temp_C;
const deskripsi = data.current_condition[0].weatherDesc[0].value;

// Tulis ke elemen HTML
document.getElementById('suhu-saat-ini').innerText = suhu;
document.getElementById('deskripsi-cuaca').innerText = deskripsi;

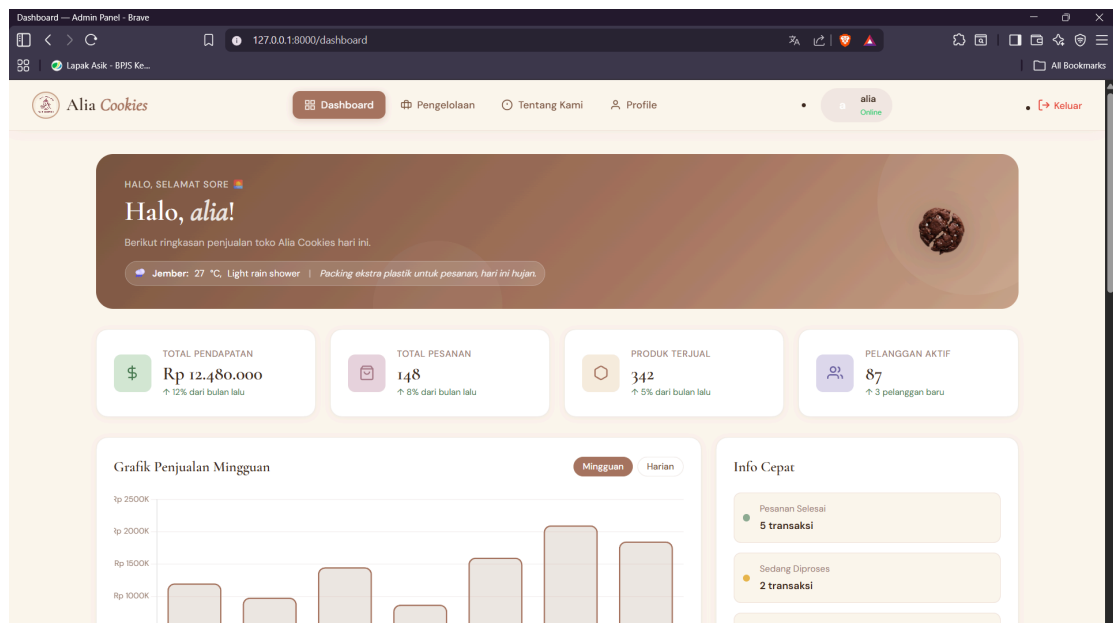
// Logika bisnis kemasan kue Alia Cookies berdasarkan cuaca
const rekomendasi = document.getElementById('rekomendasi-toko');
const descLower = deskripsi.toLowerCase();

if (descLower.includes('rain') || descLower.includes('shower') || descLower.includes('drizzle')) {
rekomendasi.innerText = "Packing ekstra plastik untuk J&T, hari ini hujan.";
if(document.getElementById('weather-icon')) document.getElementById('weather-icon').innerText = "☁️";
} else {
rekomendasi.innerText = "Aman untuk pengiriman reguler hampers.";
if(document.getElementById('weather-icon')) document.getElementById('weather-icon').innerText = "☀️";
}

// Hilangkan loading indicator dan munculkan data asli
loading.style.display = 'none';
konten.style.display = 'flex';
} catch (error) {
console.error(error);
loading.innerText = "❌ Gagal memuat info cuaca";
}
}

```

Tampilan pada dashboard admin:



2. Buat live search yang memiliki form sederhana untuk mencari mahasiswa. Form dikirim via AJAX POST/ GET ke Laravel — halaman tidak boleh reload sama

sekali. Setelah berhasil, catatan baru langsung muncul di daftar bawah tanpa refresh. Gunakan CSRF token yang benar.

Langkah 1: Membuat File Potongan Tabel

(resources/views/partials/transaction_table.blade.php)

Tabel transaksi dipotong dan dipisahkan menjadi *file* mandiri bernama `transaction_table.blade.php`. Hal ini dilakukan agar server hanya merender ulang bagian tabelnya saja saat pencarian, bukan seluruh halaman HTML.

```
1 <div class="table-responsive">
2   <table class="data-table" id="trxTable">
3     <thead>
4       <tr>
5         <th>ID Transaksi</th>
6         <th>Pelanggan</th>
7         <th>Produk</th>
8         <th>Status</th>
9         <th>Total</th>
10        <th>Tanggal</th>
11      </tr>
12    </thead>
13    <tbody>
14      @forelse($transactions as $trx)
15        <tr>
16          <td><span class="trx-id">{{ $trx->id }}</span></td>
17          <td>{{ $trx->user ? $trx->user->name : 'Pelanggan Umum' }}</td>
18          <td>{{ $trx->produk_ringkasan }}</td>
19          <td>
20            <span class="status-badge status-{{ strtolower($trx->status_pesanan) }}">
21              {{ $trx->status_pesanan }}
22            </span>
23          </td>
24          <td class="td-money">Rp {{ number_format($trx->total, 0, ',', '.') }}</td>
25          <td class="td-date">{{ $trx->tanggal_transaksi }}</td>
26        </tr>
27      @empty
28        <tr>
29          <td colspan="6" style="text-align: center; color: var(--text-light); padding: 20px;">Data transaksi tidak di-
30        </td>
31      @endforelse
32    </tbody>
33  </table>
34 </div>
35
36 <div class="pagination-wrapper" style="margin-top: 20px; display: flex; justify-content: flex-end;">
37   {{ $transactions->links() }}
38 </div>
39
```

Langkah 2: Menyesuaikan Query di DashboardController.php

Pada `DashboardController`, ditambahkan pengecekan *request* AJAX. Jika terdeteksi kata kunci (keyword), sistem akan memfilter *database* menggunakan klausa *where* dan *orWhereHas* (untuk relasi tabel *user* dan *items*), lalu mereturn data spesifik ke *view* parsial.

```

public function index(Request $request)
{
    if (!session()->has('success')) {
        session()->flash('success', 'Selamat datang kembali di Panel Admin Alia Cookies!');
    }

    $query = Transaction::with(['user', 'items']);

    if ($request->has('keyword') && $request->get('keyword') != '') {
        $keyword = $request->get('keyword');
        $query->where(function($q) use ($keyword) {
            $q->where('id', 'LIKE', "%{$keyword}%")

            ->orWhereHas('user', function($userQuery) use ($keyword) {
                $userQuery->where('name', 'LIKE', "%{$keyword}%");
            })

            ->orWhereHas('items', function($subQuery) use ($keyword) {
                $subQuery->where('nama_produk', 'LIKE', "%{$keyword}%");
            })
        });
    }

    $transactions = $query->latest('tanggal_transaksi')->paginate(10);

    if ($request->ajax()) {
        return view('partials.transaction_table', compact('transactions'))->render();
    }
}

```

Langkah 3: Memasang Kontainer Tukar Data di dashboard.blade.php

Penerapan code:

```

<div id="table-container">
    @include('partials.transaction_table')
</div>

```

Fungsi javascript:

JavaScript dipasang untuk mendengarkan setiap ketikan di kolom pencarian. Data dikirim secara asinkron ke server, lalu balasan dari server akan langsung menggantikan struktur HTML lama di dalam `<div id="table-container">`.

```
// Fungsi Utama Live Search AJAX
async function doLiveSearch(keyword) {
  const container = document.getElementById('table-container');
  try {
    const response = await fetch(`/dashboard?keyword=${encodeURIComponent(keyword)}`, {
      headers: { 'X-Requested-With': 'XMLHttpRequest' }
    });
    const html = await response.text();
    container.innerHTML = html;
  } catch (error) {
    console.error("Gagal memuat hasil pencarian Live Search:", error);
  }
}

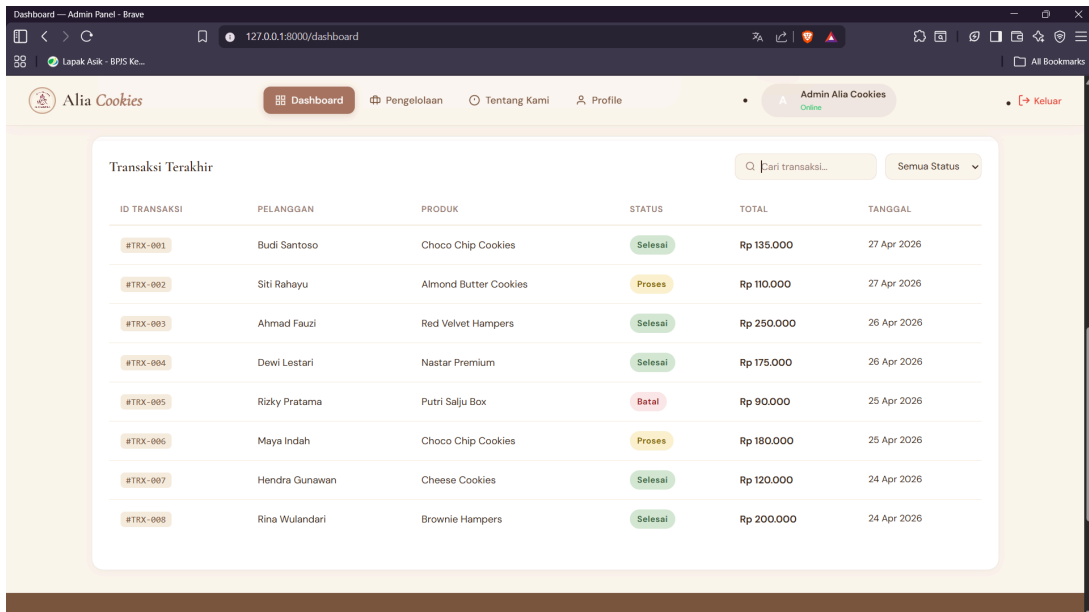
// Interceptor Tombol Pagination Laravel agar Tidak Reload Halaman
document.addEventListener('click', async function(e) {
  const paginationLink = e.target.closest('.pagination a');
  if (paginationLink) {
    e.preventDefault();

    const url = paginationLink.href;
    const keyword = document.getElementById('searchTrx').value;
    const container = document.getElementById('table-container');

    try {
      const targetUrl = new URL(url);
      if(keyword) targetUrl.searchParams.set('keyword', keyword);

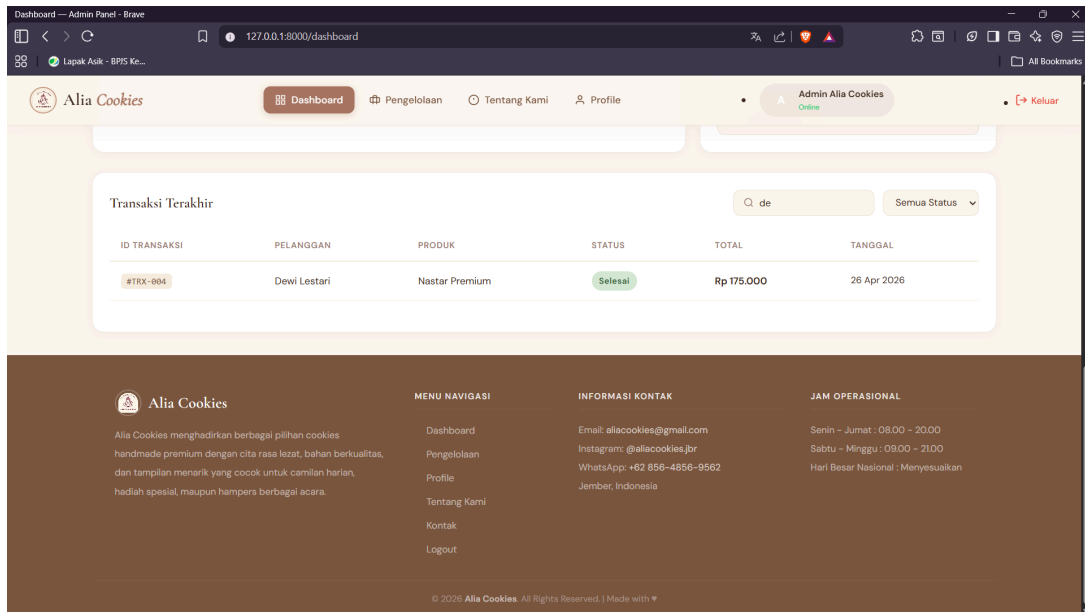
      const response = await fetch(targetUrl, {
        headers: { 'X-Requested-With': 'XMLHttpRequest' }
      });
      const html = await response.text();
      container.innerHTML = html;
    } catch (error) {
      console.error("Gagal memuat halaman pagination AJAX:", error);
    }
  }
});
```

Tampilan:



The screenshot shows the 'Admin Panel - Brave' browser window displaying the 'Dashboard' of 'Alia Cookies'. The page features a navigation bar with links for 'Dashboard', 'Pengelolaan', 'Tentang Kami', and 'Profile'. A user profile dropdown for 'Admin Alia Cookies' is visible. The main content area is titled 'Transaksi Terakhir' and includes a search bar and a 'Semua Status' filter. Below this is a table with 8 columns: ID TRANSAKSI, PELANGGAN, PRODUK, STATUS, TOTAL, and TANGGAL. The table lists 8 transactions with their respective details and status (Selesai, Proses, or Batal).

ID TRANSAKSI	PELANGGAN	PRODUK	STATUS	TOTAL	TANGGAL
#TRX-001	Budi Santoso	Choco Chip Cookies	Selesai	Rp 135.000	27 Apr 2026
#TRX-002	Siti Rahayu	Almond Butter Cookies	Proses	Rp 110.000	27 Apr 2026
#TRX-003	Ahmad Fauzi	Red Velvet Hampers	Selesai	Rp 250.000	26 Apr 2026
#TRX-004	Dewi Lestari	Nastar Premium	Selesai	Rp 175.000	26 Apr 2026
#TRX-005	Rizky Pratama	Putri Salju Box	Batal	Rp 90.000	25 Apr 2026
#TRX-006	Maya Indah	Choco Chip Cookies	Proses	Rp 180.000	25 Apr 2026
#TRX-007	Hendra Gunawan	Cheese Cookies	Selesai	Rp 120.000	24 Apr 2026
#TRX-008	Rina Wulandari	Brownie Hampers	Selesai	Rp 200.000	24 Apr 2026



3. Buat navbar untuk mengeset dark mode.

- a. Tambahkan dark mode ke konfigurasi Tailwind (darkMode: 'class' di [tailwind.config.js](#))

Fitur darkMode: 'class' diaktifkan pada Tailwind. Di sisi CSS murni (style.css), ditambahkan pemilih (selector) khusus html.dark untuk mengganti *background* dan warna teks menjadi skema coklat gelap jika mode aktif.

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  darkMode: 'class',
  content: [
    './resources/**/*.blade.php',
    './resources/**/*.js',
    './resources/**/*.vue',
  ],
  theme: {
    extend: {
      colors: {
        cream: { light: '#faf5ef', dark: '#f3e8dc' },
        chocolate: '#5a453a'
      }
    }
  },
  plugins: [],
}
```

- b. Buat helper function JavaScript: setCookie, getCookie, deleteCookie

Agar layar tidak berkedip terang saat halaman baru dimuat, sebuah skrip fungsi pembaca cookie (getCookieInHead) disematkan di dalam tag <head> HTML. Skrip ini akan membaca memori secara instan sebelum seluruh halaman selesai dirender.

```

<script>
function setCookie(name, value, days = 30) {
    let expires = "";
    if (days) {
        let date = new Date();
        date.setTime(date.getTime() + (days * 24 * 60 * 60 * 1000));
        expires = "; expires=" + date.toUTCString();
    }
    document.cookie = name + "=" + encodeURIComponent(value) + expires + "; path=/; SameSite=Lax";
}

function getCookie(name) {
    let match = document.cookie.match(new RegExp('(^| )' + name + '=[^;]+'));
    if (match) return decodeURIComponent(match[2]);
    return null;
}

function deleteCookie(name) {
    document.cookie = name + "=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;";
}
</script>

```

c. Implementasikan toggle dark mode: klik tombol → toggle class 'dark' di HTML → simpan ke cookie

Terdapat tombol saklar di *navbar* yang memicu fungsi JavaScript `toggleDarkMode()`. Fungsi ini bertugas menghidupkan/mematikan *class* 'dark' di HTML secara langsung (*real-time*) sekaligus memperbarui nilai *cookie*.

Langkah 1: Pasang Tombol di Navbar

```

<li>
<a href="javascript:void(0)" onclick="toggleDarkMode()" style="display: flex; align-items: center; gap: 8px;">
    <svg id="moon-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" style="width: 18px;" />
    <svg id="sun-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" style="width: 18px;" />
    <span id="theme-text">Mode Gelap</span>
</a>
</li>

```

Langkah 2: Tambahkan Fungsi `toggleDarkMode` di `app.blade.php`

```

function toggleDarkMode() {
    const isDarkNow = document.documentElement.classList.toggle('dark');
    setCookie('theme', isDarkNow ? 'dark' : 'light', 30);

    const moonIcon = document.getElementById('moon-icon');
    const sunIcon = document.getElementById('sun-icon');
    const themeText = document.getElementById('theme-text');

    if (isDarkNow) {
        if(moonIcon) moonIcon.style.display = 'none';
        if(sunIcon) sunIcon.style.display = 'inline-block';
        if(themeText) themeText.innerText = 'Mode Terang';
    } else {
        if(moonIcon) moonIcon.style.display = 'inline-block';
        if(sunIcon) sunIcon.style.display = 'none';
        if(themeText) themeText.innerText = 'Mode Gelap';
    }
}

```

d. Terapkan cookie di awal load (dalam tag script di <head>) untuk mencegah flash of unstyled content


```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>@yield('title', 'Alia Cookies') – Admin Panel</title>
  <script>
    function getCookieInHead(name) {
      let match = document.cookie.match(new RegExp('(\\| )' + name + '=[^;]+'));
      if (match) return decodeURIComponent(match[2]);
      return null;
    }

    const savedTheme = getCookieInHead('theme') || 'system';

    const systemPrefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;

    if (savedTheme === 'dark' || (savedTheme === 'system' && systemPrefersDark)) {
      document.documentElement.classList.add('dark');
    } else {
      document.documentElement.classList.remove('dark');
    }
  </script>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

  <link href="https://fonts.googleapis.com/css2?family=Cormorant+Garamond:ital,wght@0,400;0,5

  <link rel="stylesheet" href="{{ asset('css/style.css') }}">

  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>

```

e. **Buat halaman pengaturan /preferensi dengan form: pilihan tema (light/dark/system), pilihan ukuran font**

Untuk kustomisasi yang lebih lengkap, disediakan halaman /preferensi yang berisi *form* pilihan tema dan ukuran font. Saat *form* disubmit, data dikirim tanpa *reload* menggunakan Fetch API menuju PreferenceController.php. Controller ini kemudian membungkus pilihan tersebut ke dalam Cookie Laravel agar preferensi admin tersimpan secara permanen.

Langkah 1: Daftarkan Jalur Halaman di routes/web.php

```

Route::get('/preferensi', function () {
  return view('preferensi');
})->name('preferensi.index');

```

Langkah 2: Buat File View Baru (resources/views/preferensi.blade.php)

Langkah 3: Tambahkan Link Menu di navbar.blade.php

```
<li>
  <a href="{{ route('preferensi.index') }}" class="{{ request()->routeIs('preferensi.index') ? 'active' : '' }}">
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="12" cy="12" r="3"/><path d="M12 14.5 12 12 14.5 12" /></svg>
    Preferensi
  </a>
</li>
```

f. Simpan form ke cookie via Fetch POST ke endpoint Laravel

```
@push('scripts')
<script>
  document.getElementById('preferencesForm').addEventListener('submit', async function(e) {
    // 1. Hentikan aksi bawaan form HTML agar tidak refresh halaman
    e.preventDefault();

    // 2. Ambil semua data inputan dari Form (theme dan font_size)
    const formData = new FormData(this);

    try {
      // 3. Kirim data secara asynchronous (Fetch API) dengan metode POST
      const response = await fetch("{{ route('preferensi.store') }}", {
        method: "POST",
        headers: {
          'X-Requested-With': 'XMLHttpRequest',
          'X-CSRF-TOKEN': formData.get('_token') // Ambil token CSRF untuk keamanan Laravel
        },
        body: formData
      });

      // 4. Ubah respon mentah dari Laravel menjadi format JSON digital
      const result = await response.json();

      // 5. Jika response sukses, tampilkan notifikasi dan muat ulang halaman
      if (result.status === 'success') {
        alert(result.message);

        // Reload halaman agar cookie baru langsung dibaca oleh skrip anti-flash di <head>
        window.location.reload();
      }
    } catch (error) {
      console.error("Gagal mengirim preferensi ke server:", error);
      alert("Terjadi kesalahan saat menyimpan pengaturan.");
    }
  });
</script>
@endpush
```

g. Laravel Controller: baca cookie dari \$request->cookie(), kembalikan JSON konfirmasi

Langkah 1: Daftarkan Rute POST di routes/web.php

```
Route::post('/preferensi', [PreferenceController::class, 'store'])->name('preferensi.store');
```

Langkah 2: Buat Berkas PreferenceController.php

```
PS C:\Users\HP\OneDrive\ドキュメント\AliaCookies\AliaCookies_Project_PWEB\tugas-laravel> php artisan make:controller PreferenceController
INFO: Controller [C:\Users\HP\OneDrive\ドキュメント\AliaCookies\AliaCookies_Project_PWEB\tugas-laravel\app\Http\Controllers\PreferenceController.php] created successfully.
```

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Cookie;

class PreferenceController extends Controller
{
    public function store(Request $request): JsonResponse
    {
        $currentThemeCookie = $request->cookie('theme');
        $currentFontCookie = $request->cookie('font_size');

        $themeSelected = $request->input('theme', 'system');
        $fontSelected = $request->input('font_size', 'md');

        $cookieTheme = Cookie::make('theme', $themeSelected, 43200, null, null, false, false);
        $cookieFont = Cookie::make('font_size', $fontSelected, 43200, null, null, false, false);

        return response()->json([
            'status' => 'success',
            'message' => 'Konfigurasi preferensi tampilan Alia Cookies berhasil diperbarui!',
            'theme' => $themeSelected,
            'font_size' => $fontSelected
        ])->withCookie($cookieTheme)->withCookie($cookieFont);
    }
}
```

Setelah semua step sudah dilakukan, saatnya mengatur tampilan darkmode pada style.css. Berikut tampilan beberapa kodenya:

```

/* DARK MODE */
html.dark {
  --dm-bg: #1c1410;
  --dm-bg-card: #ffebda;
  --dm-bg-input: #2d241f;
  --dm-bg-hover: #3a2d26;
  --dm-bg-navbar: #251c18;
  --dm-bg-table-th: #2a201a;
  --dm-bg-row-alt: #231a15;
  --dm-text-primary: #3d2b1f;
  --dm-text-body: #fff8f2;
  --dm-text-muted: #c4a99a;
  --dm-text-label: #a77761;
  --dm-text-code: #f0c9a0;
  --dm-border: rgba(167, 119, 97, 0.25);
  --dm-border-card: #e6ccb2;
  --dm-mocha: #c9a48a;
  --dm-green: #7ec986;
  --dm-yellow: #e8c56b;
  --dm-red: #e88080;
}

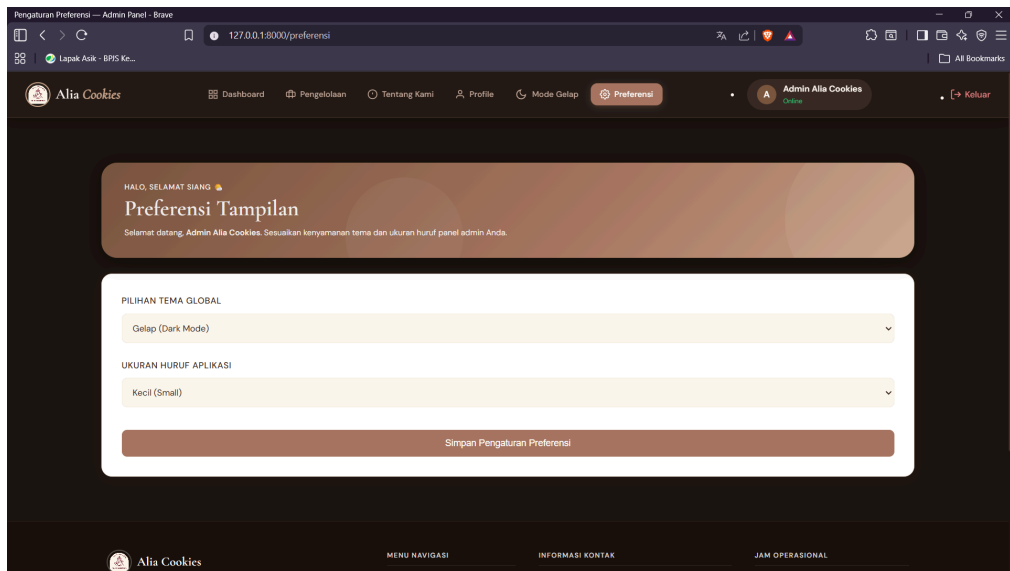
html.dark body {
  background-color: var(--dm-bg);
  color: var(--dm-text-body);
}

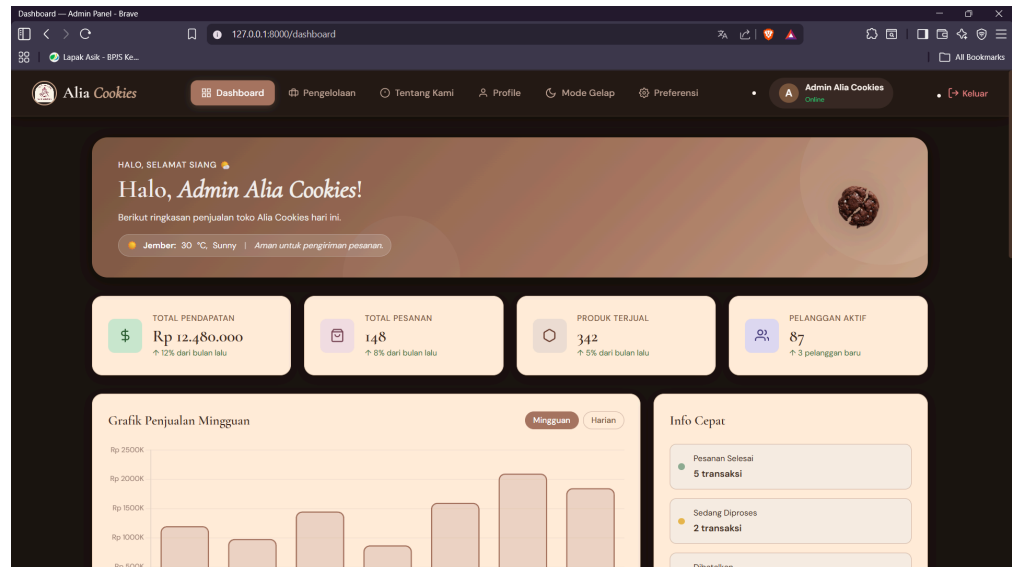
html.dark .main-content {
  background-color: var(--dm-bg);
}

html.dark .navbar {
  background: rgba(37, 28, 24, 0.95);
  backdrop-filter: blur(16px);
  border-bottom: 1px solid var(--dm-border);
  box-shadow: 0 2px 20px rgba(0, 0, 0, 0.3);
}

```

Hasil akhir untuk darkmode:





4. **Buat section yang menghitung berapa kali pengguna mengunjungi halaman tersebut menggunakan session Laravel. Tampilkan: jumlah kunjungan, waktu kunjungan pertama, dan waktu kunjungan terakhir. Sediakan tombol "Reset Hitungan" yang menghapus data dari session dan mengulang dari awal.**

Langkah 1: Atur Logika Session di Controller

Di dalam fungsi index pada DashboardController, sistem ditugaskan untuk membaca dan memanipulasi *Session*. Kunjungan pertama (*first_visit*) disimpan hanya jika session tersebut kosong. Kemudian, total kunjungan (*visit_count*) ditambahkan +1 setiap kali halaman dibuka, dan waktu kunjungan terakhir (*last_visit*) selalu diperbarui dengan waktu saat itu (*now()*).

```

class DashboardController extends Controller
{
    public function index(Request $request)
    {
        if (!session()->has('success')) {
            session()->flash('success', 'Selamat datang kembali di Panel Admin Alia Cookies!');
        }

        $visitCount = $request->session()->get('visit_count', 0) + 1;
        $request->session()->put('visit_count', $visitCount);

        if (!$request->session()->has('first_visit')) {
            $request->session()->put('first_visit', now());
        }

        $request->session()->put('last_visit', now());

        $query = Transaction::with(['user', 'items']);

        if ($request->has('keyword') && $request->get('keyword') != '') {
            $keyword = $request->get('keyword');
            $query->where(function($q) use ($keyword) {
                $q->where('id', 'LIKE', "%{$keyword}%")
                    ->orWhereHas('user', function($userQuery) use ($keyword) {
                        $userQuery->where('name', 'LIKE', "%{$keyword}%");
                    })
                    ->orWhereHas('items', function($subQuery) use ($keyword) {
                        $subQuery->where('nama_produk', 'LIKE', "%{$keyword}%");
                    });
            });
        }

        $transactions = $query->latest('tanggal_transaksi')->paginate(10);

        if ($request->ajax()) {
            return view('partials.transaction_table', compact('transactions'))->render();
        }
    }
}

```

```

public function resetKunjungan(Request $request)
{
    $request->session()->forget(['visit_count', 'first_visit', 'last_visit']);

    return redirect()->back()->with('success', 'Hitungan kunjungan berhasil direset dari awal!');
}

```

Langkah 2: Daftarkan Route untuk Tombol Reset

Untuk menghapus riwayat, dibuat sebuah fungsi baru bernama `resetKunjungan` di dalam *controller* yang berisi perintah `$request->session()->forget([...])`. Fungsi ini bertugas membersihkan seluruh kunci sesi kunjungan. Sebuah *route* berjenis POST kemudian didaftarkan agar fungsi ini bisa dipanggil.

```

Route::post('/reset-kunjungan', [DashboardController::class, 'resetKunjungan']->name('kunjungan.reset'));

```

Langkah 3: Tampilkan di File View (Blade)

Data hitungan kunjungan, waktu kunjungan pertama, dan waktu kunjungan terakhir yang telah diproses di *controller* dikirim ke *file* `dashboard.blade.php` untuk ditampilkan ke layar. Di bagian bawah informasi tersebut, ditambahkan sebuah *form* dengan tombol "Reset Hitungan" yang mengarah ke *route* pembersihan data.

```

<div style="display: flex; flex-direction: column; gap: 8px;">
  <div class="stat-card stat-mocha" style="background-color: #ffffff; border: 1px solid #e6ccb2; border-radius: 12px; padding: 12px; text-align: center;">
    <div class="stat-icon">
      <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 8 8 12 12 16 16 12 12 8" /></svg>
    </div>
    <div class="stat-info">
      <span class="stat-label">Total Kunjungan</span>
      <span class="stat-value">{{ $visitCount }}</span>
      <span class="stat-change up" style="color: #4a7c51; font-weight: bold; font-size: 0.8em;">↑ Pk1 {{ \Carbon\Carbon::parse($lastVisit)->format('d M Y')}}</span>
    </div>
  </div>

  <form action="{{ route('kunjungan.reset') }}" method="POST" style="margin: 0;">
    @csrf
    <button type="submit" style="width: 100%; font-size: 11px; padding: 6px 12px; border-radius: 8px; border: 1px solid #e6ccb2; background-color: #ffffff; color: #4a7c51;">
      <strong>🔄 Reset Data</strong>
      <span style="opacity: 0.8; font-size: 0.8em; margin-left: 5px;">Awal: {{ \Carbon\Carbon::parse($firstVisit)->translatedFormat('d M Y')}}</span>
    </button>
  </form>
</div>

```

Hasil tampilan pada dashboard

